Assignment #1

Embodied Artificial Intelligence (CS1050)

Name: Jiang Jiahao **Student ID:** 22671064

Instructor: Professor Gaoang Wang **Term:** Fall 2026

Due: October 30, 2026, 23:59 (China Standard Time)

Question 1

Solution

The two lines have the common direction $\mathbf{d} = (4, 2, 5)^\top$. For a point $\mathbf{a} + \lambda\mathbf{d}$ on either line,

$$\lim_{\lambda\to\infty}\left(\frac{a_X + 4\lambda}{a_Z + 5\lambda}, \frac{a_Y + 2\lambda}{a_Z + 5\lambda}\right) = \left(\frac{4}{5}, \frac{2}{5}\right).$$

Thus both lines have the same vanishing point $\boxed{\mathbf{v} = (4/5,\ 2/5)^\top = (0.8,\ 0.4)^\top}$. Its homogeneous image coordinates are $(4, 2, 5)^\top$; the different line origins do not affect this limit.

Python code

`experients/assignment1/q1_vanishing_point.py`

```python
"""Q1: compute both vanishing points as exact symbolic limits."""
import sympy as sp


def vanishing_points():
    lam = sp.symbols("lambda", positive=True)
    direction = sp.Matrix([4, 2, 5])
    origins = (sp.Matrix([3, 1, 6]), sp.Matrix([7, 4, 2]))
    points = []
    for origin in origins:
        point = origin + lam * direction
        points.append(tuple(sp.limit(point[j] / point[2], lam, sp.oo)
                            for j in (0, 1)))
    return points


if __name__ == "__main__":
    q, r = vanishing_points()
    print("Q vanishing point:", q)
    print("R vanishing point:", r)
    print("Same vanishing point:", q == r)
```

Run output

```
Q vanishing point: (4/5, 2/5)
R vanishing point: (4/5, 2/5)
Same vanishing point: True
```

Question 2

Solution

The plane has normal $\mathbf{n} = (6, 3, 2)^\top$. Every direction $\mathbf{d}$ parallel to it satisfies $\mathbf{n}^\top \mathbf{d} = 0$. Under normalized perspective projection, its vanishing point is $\mathbf{v} \sim (d_X, d_Y, d_Z)^\top$. Therefore all these points satisfy

$$\mathbf{n}^\top \mathbf{v} = 0 \quad \Longrightarrow \quad \boxed{6x + 3y + 2 = 0}, \qquad \boxed{\ell_\infty \sim (6, 3, 2)^\top}.$$

The offset 17 determines the plane's position, whereas its vanishing line depends only on its orientation. Directions with $d_Z = 0$ are included as homogeneous points at infinity.

Python code

experients/assignment1/q2_vanishing_line.py

```python
"""Q2: obtain the vanishing line from the plane normal."""
import sympy as sp


def vanishing_line():
    X, Y, Z, x, y = sp.symbols("X Y Z x y")
    plane = 6 * X + 3 * Y + 2 * Z - 17
    normal = sp.Matrix([sp.diff(plane, v) for v in (X, Y, Z)])
    line = sp.Eq(normal.dot(sp.Matrix([x, y, 1])), 0)
    return normal, line


if __name__ == "__main__":
    normal, line = vanishing_line()
    print("Plane normal:", tuple(normal))
    print("Vanishing line:", line)
    print("Homogeneous line coefficients:", tuple(normal))
```

Run output

```
Plane normal: (6, 3, 2)
Vanishing line: Eq(6*x + 3*y + 2, 0)
Homogeneous line coefficients: (6, 3, 2)
```

Question 3

Solution

For $12X + 8Y + 28Z = 23$, a normal is

$$\mathbf{n} = (12, 8, 28)^\top = 4(3, 2, 7)^\top.$$

If $\mathbf{u}$ and $\mathbf{v}$ lie on the plane, subtracting their plane equations gives $\mathbf{n}^\top(\mathbf{u} - \mathbf{v}) = 0$. Consequently $(3, 2, 7)^\top$ is orthogonal to every direction in the plane, so it is perpendicular to the plane.

Python code

experients/assignment1/q3_plane_normal.py

```python
"""Q3: check proportional normals and a basis of plane directions."""
import sympy as sp


def perpendicular_check():
    X, Y, Z = sp.symbols("X Y Z")
    plane = 12 * X + 8 * Y + 28 * Z - 23
    normal = sp.Matrix([sp.diff(plane, v) for v in (X, Y, Z)])
    vector = sp.Matrix([3, 2, 7])
    scale = normal[0] / vector[0]
    directions = normal.T.nullspace()
    dot_products = [vector.dot(d) for d in directions]
    perpendicular = normal == scale * vector and dot_products == [0, 0]
    return normal, scale, dot_products, perpendicular


if __name__ == "__main__":
    normal, scale, dot_products, perpendicular = perpendicular_check()
    print("Plane normal:", tuple(normal))
    print("Normal / given vector scale:", scale)
    print("Dot products with two independent plane directions:", dot_products)
    print("Given vector is perpendicular to the plane:", perpendicular)
```

Run output

```
Plane normal: (12, 8, 28)
Normal / given vector scale: 4
Dot products with two independent plane directions: [0, 0]
Given vector is perpendicular to the plane: True
```

Question 4

Solution

The shared translation $\mathbf{t} = (-3, 2, 1)^\top$ cancels: $\mathbf{x}_2 - \mathbf{y}_2 = R(\mathbf{x}_1 - \mathbf{y}_1)$. Writing $\mathbf{d} = \mathbf{x}_1 - \mathbf{y}_1$ and using $R^\top R = I$,

$$\|\mathbf{x}_2 - \mathbf{y}_2\|_2^2 = \mathbf{d}^\top R^\top R \mathbf{d} = \mathbf{d}^\top \mathbf{d} = \|\mathbf{x}_1 - \mathbf{y}_1\|_2^2.$$

Both norms are nonnegative; taking square roots proves $\boxed{\|\mathbf{x}_2 - \mathbf{y}_2\|_2 = \|\mathbf{x}_1 - \mathbf{y}_1\|_2}$. The code verifies the identity symbolically under the same orthogonality assumption; it does not rely on selecting a particular rotation.

Python code

experiments/assignment1/q4_rigid_distance.py

```python
"""Q4: symbolically verify distance preservation for an orthogonal R."""
import sympy as sp


def distance_check():
    R = sp.MatrixSymbol("R", 3, 3)
    x, y = (sp.MatrixSymbol(name, 3, 1) for name in ("x", "y"))
    t = sp.Matrix([-3, 2, 1])
    difference = (R * x + t) - (R * y + t)
    cancellation = (difference - R * (x - y)).as_explicit()
    cancels = cancellation.applyfunc(sp.simplify) == sp.zeros(3, 1)
    # Orthogonality is the given rotation-matrix assumption.
    gram = sp.refine(R.T * R, sp.Q.orthogonal(R)).doit()
    d = sp.MatrixSymbol("d", 3, 1)
    change = (d.T * (gram.as_explicit() - sp.eye(3)) * d)[0, 0]
    return cancels, gram, sp.simplify(change)


if __name__ == "__main__":
    cancels, gram, change = distance_check()
    print("The common translation cancels:", cancels)
    print("R.T * R for a rotation matrix:", gram)
    print("Change in squared distance:", change)
    print("Euclidean distances are equal:", cancels and change == 0)
```

Run output

```
The common translation cancels: True
R.T * R for a rotation matrix: I
Change in squared distance: 0
Euclidean distances are equal: True
```

Question 5

Solution

Let $\mathbf{X}_i = (X_i, Y_i, Z_i, 1)^\top$ and denote the rows of P by $\mathbf{p}_1^\top, \mathbf{p}_2^\top, \mathbf{p}_3^\top$. Each correspondence imposes the two linear constraints

$$(\mathbf{p}_1^\top - x_i \mathbf{p}_3^\top)\mathbf{X}_i = 0, \qquad (\mathbf{p}_2^\top - y_i \mathbf{p}_3^\top)\mathbf{X}_i = 0.$$

Stacking the six pairs gives $A\mathbf{p} = 0$, where $\mathbf{p}$ contains the rows of P in order. The 12×12 matrix A has rank 11, so the solution is unique up to scale. Its exact representative and norm are

$$P_0 = \begin{bmatrix} 180 & 0 & 0 & 280 \\ 0 & 180 & 0 & 220 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \|P_0\|_F = \sqrt{191601}.$$

The requested normalization is

$$\boxed{P = \frac{1}{\sqrt{191601}} \begin{bmatrix} 180 & 0 & 0 & 280 \\ 0 & 180 & 0 & 220 \\ 0 & 0 & 0 & 1 \end{bmatrix}}.$$

This is an affine camera: $x = 180X + 280$ and $y = 180Y + 220$ do not depend on Z. The code uses normalized DLT: it centers and scales both point sets, takes the last right singular vector, and denormalizes by $P = T_{2D}^{-1} P_n T_{3D}$. It then enforces $\|P\|_F = 1$ and fixes the global sign by making the largest-magnitude entry positive.

Python code

`experients/assignment1/q5_camera.py`

```python
"""Question 5: normalized 3D-to-2D DLT, using the given six columns."""

import numpy as np


WORLD = np.array([
    [0, 1, 0, 1, 0.5, 0.5],
    [0, 0, 1, 1, 0.5, 0],
    [2, 2, 2, 2, 4, 4],
], dtype=float)
IMAGE = np.array([
    [280, 460, 280, 460, 370, 370],
    [220, 220, 400, 400, 310, 220],
], dtype=float)


def normalize_points(points):
    """Center points and set mean distance to sqrt(dimension)."""
    dimension = points.shape[0]
    center = points.mean(axis=1)
    distance = np.linalg.norm(points - center[:, None], axis=0).mean()
    if distance <= np.finfo(float).eps:
        raise ValueError("Coincident points cannot be normalized.")
    scale = np.sqrt(dimension) / distance
    transform = np.eye(dimension + 1)
    transform[:dimension, :dimension] *= scale
    transform[:dimension, -1] = -scale * center
    homogeneous = np.vstack((points, np.ones(points.shape[1])))
    return transform @ homogeneous, transform


```

```python
def estimate_camera(world=WORLD, image=IMAGE):
    """Solve A p = 0, denormalize, fix sign, and set ||P||_F = 1."""
    if world.shape != (3, 6) or image.shape != (2, 6):
        raise ValueError("This exercise expects six 3D/2D correspondences.")
    world_n, world_transform = normalize_points(world)
    image_n, image_transform = normalize_points(image)
    rows = []
    for point, pixel in zip(world_n.T, image_n.T):
        u, v = pixel[:2] / pixel[2]
        rows.append(np.r_[point, np.zeros(4), -u * point])
        rows.append(np.r_[np.zeros(4), point, -v * point])
    design = np.array(rows)
    _, singular_values, vh = np.linalg.svd(design)
    rank = np.linalg.matrix_rank(design)
    if rank != 11:
        raise ValueError(f"Expected a unique camera up to scale; rank={rank}.")
    camera_n = vh[-1].reshape(3, 4)
    camera = np.linalg.solve(image_transform, camera_n @ world_transform)
    camera /= np.linalg.norm(camera, ord="fro")
    if camera.flat[np.argmax(np.abs(camera))] < 0:
        camera = -camera
    homogeneous = np.vstack((world, np.ones(world.shape[1])))
    projected_h = camera @ homogeneous
    if np.any(np.abs(projected_h[2]) < 1e-12):
        raise ValueError("A supplied point projects to infinity.")
    projected = projected_h[:2] / projected_h[2]
    errors = np.linalg.norm(projected - image, axis=0)
    return camera, projected, errors, singular_values, rank


def main():
    np.set_printoptions(precision=10, suppress=True, linewidth=90)
    camera, projected, errors, _, rank = estimate_camera()
    print("P (Frobenius norm = 1):")
    print(camera)
    print(f"Frobenius norm: {np.linalg.norm(camera, 'fro'):.12f}")
    print(f"Normalized design matrix rank: {rank} / 12")
    print("Reprojected pixels (columns match the assignment):")
    print(projected)
    print(f"Maximum reprojection error (px): {errors.max():.3e}")


if __name__ == "__main__":
    main()
```

Run output

```
P (Frobenius norm = 1):
[[ 0.411219423  -0.           -0.            0.639674658 ]
 [ 0.            0.411219423  -0.            0.502601517 ]
 [ 0.           -0.           -0.            0.0022845524]]
Frobenius norm: 1.000000000000
Normalized design matrix rank: 11 / 12
Reprojected pixels (columns match the assignment):
[[280. 460. 280. 460. 370. 370.]
 [220. 220. 400. 400. 310. 220.]]
Maximum reprojection error (px): 1.798e-13
```

Question 6

Solution

Let $\mathbf{X}_h = (X, Y, Z, W)^\top$. For each camera, stack the rows $x_j P_3^{(j)} - P_1^{(j)}$ and $y_j P_3^{(j)} - P_2^{(j)}$. The supplied matrices and pixels give

$$A = \begin{bmatrix} -900 & 0 & 0 & 0 \\ 0 & -900 & 0 & 0 \\ -900 & 0 & -15 & 6 \\ 0 & -900 & 0 & 0 \end{bmatrix}, \qquad A\mathbf{X}_h = 0.$$

The first two equations imply $X = Y = 0$; the third then gives $-15Z + 6W = 0$. Choosing $W = 1$ yields

$$\boxed{\mathbf{X} = (0,\ 0,\ 0.4)^\top}, \qquad \mathbf{X}_h = (0,\ 0,\ 0.4,\ 1)^\top.$$

Writing $P^{(2)} = K[I \mid \mathbf{t}]$ gives $\mathbf{t} = (-6/900, 0, 0)^\top$ and camera center $\mathbf{C}_2 = (6/900, 0, 0)^\top$. The baseline is $B = 6/900 = 1/150$ in the given world units. The depth can also be checked from the disparity:

$$Z = \frac{fB}{x_1 - x_2} = \frac{900(6/900)}{400 - 385} = 0.4.$$

The code solves the homogeneous system by SVD and divides by the last coordinate. Reprojection checks both image measurements; both depths are positive.

Python code

`experients/assignment1/q6_triangulation.py`

```python
"""Question 6: homogeneous linear triangulation with the supplied matrices."""

import numpy as np


P1 = np.array([
    [900, 0, 400, 0],
    [0, 900, 250, 0],
    [0, 0, 1, 0],
], dtype=float)
P2 = np.array([
    [900, 0, 400, -6],
    [0, 900, 250, 0],
    [0, 0, 1, 0],
], dtype=float)
PIXEL1 = np.array([400, 250], dtype=float)
PIXEL2 = np.array([385, 250], dtype=float)


def triangulate(p1=P1, p2=P2, pixel1=PIXEL1, pixel2=PIXEL2):
    """Stack x P_3 - P_1 and y P_3 - P_2 for each view."""
    design = np.vstack([
        pixel1[0] * p1[2] - p1[0],
        pixel1[1] * p1[2] - p1[1],
        pixel2[0] * p2[2] - p2[0],
        pixel2[1] * p2[2] - p2[1],
    ])
    _, singular_values, vh = np.linalg.svd(design)
    if np.linalg.matrix_rank(design) != 3:
        raise ValueError("The correspondence does not determine a unique point.")
    point_h = vh[-1]
    if abs(point_h[3]) < 1e-12:
```

```
        raise ValueError("The triangulated point is at infinity.")
    point_h = point_h / point_h[3]
    projected_h = np.vstack((p1 @ point_h, p2 @ point_h))
    if np.any(np.abs(projected_h[:, 2]) < 1e-12):
        raise ValueError("Projection has a zero denominator.")
    projected = projected_h[:, :2] / projected_h[:, 2, None]
    errors = np.linalg.norm(projected - np.vstack((pixel1, pixel2)), axis=1)
    return point_h, projected, errors, singular_values


def main():
    np.set_printoptions(precision=10, suppress=True, linewidth=90)
    point_h, projected, errors, _ = triangulate()
    print("3D point [X, Y, Z]:")
    print(point_h[:3])
    print("Reprojected pixels (rows: view 1, view 2):")
    print(projected)
    print(f"Maximum reprojection error (px): {errors.max():.3e}")
    print(f"Depth in both cameras: {point_h[2]:.10f}")
    print(f"Independent disparity depth 6 / (400 - 385): {6 / 15:.10f}")


if __name__ == "__main__":
    main()
```

Run output

```
3D point [X, Y, Z]:
[0.  0.  0.4]
Reprojected pixels (rows: view 1, view 2):
[[400. 250.]
 [385. 250.]]
Maximum reprojection error (px): 2.842e-13
Depth in both cameras: 0.4000000000
Independent disparity depth 6 / (400 - 385): 0.4000000000
```

Question 7

Solution

With the convention $\mathbf{X}_2 = R\mathbf{X}_1 + \mathbf{t}$, corresponding calibrated image points obey $\mathbf{p}_2^\top E \mathbf{p}_1 = 0$, with

$$E = [\mathbf{t}]_\times R, \qquad [\mathbf{t}]_\times = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}.$$

For $R = I$ and $\mathbf{t} = (3, 0, 0)^\top$,

$$\boxed{E = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -3 \\ 0 & 3 & 0 \end{bmatrix}}.$$

It is defined up to nonzero scale. Since $K_1 = K_2 = I$, the fundamental matrix $F = K_2^{-\top} E K_1^{-1}$ is also E.

Python code

`experients/assignment1/q7_essential_matrix.py`

```python
"""Q7: E = [t]_cross R for the convention X2 = R X1 + t."""
import sympy as sp


def essential_matrix():
    R = sp.eye(3)
    tx, ty, tz = sp.Matrix([3, 0, 0])
    skew = sp.Matrix([[0, -tz, ty], [tz, 0, -tx], [-ty, tx, 0]])
    return skew * R


if __name__ == "__main__":
    E = essential_matrix()
    print("Essential matrix E:")
    for row in E.tolist():
        print(row)
    print("Rank:", E.rank())
```

Run output

```
Essential matrix E:
[0, 0, 0]
[0, 0, -3]
[0, 3, 0]
Rank: 2
```

Question 8

Solution

The epipolar line in image 2 is $\boldsymbol{\ell}_2 = E\mathbf{p}_1$:

$$\boldsymbol{\ell}_2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -3 \\ 0 & 3 & 0 \end{bmatrix} \begin{bmatrix} 280 \\ 200 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -3 \\ 600 \end{bmatrix}.$$

Thus $\boxed{\boldsymbol{\ell}_2 \sim (0, 1, -200)^\top}$, or equivalently $\boxed{y_2 = 200}$, since $0x_2 - 3y_2 + 600 = 0$.

Python code

experients/assignment1/q8_epipolar_line.py

```python
"""Q8: compute the corresponding epipolar line in image 2."""
import sympy as sp


def epipolar_line():
    tx, ty, tz = sp.Matrix([3, 0, 0])
    R = sp.eye(3)
    skew = sp.Matrix([[0, -tz, ty], [tz, 0, -tx], [-ty, tx, 0]])
    E = skew * R
    point = sp.Matrix([280, 200, 1])
    return E, E * point


if __name__ == "__main__":
    E, line = epipolar_line()
    y2 = sp.symbols("y2")
    print("Line coefficients E * p1:", tuple(line))
    print("Equivalent coefficients:", tuple(line / line[1]))
    print("Line equation:", sp.Eq(y2, sp.solve(line[1] * y2 + line[2], y2)[0]))
```

Run output

```
Line coefficients E * p1: (0, -3, 600)
Equivalent coefficients: (0, 1, -200)
Line equation: Eq(y2, 200)
```

Question 9

Solution

First multiply and then divide by the homogeneous image coordinate:

$$P\mathbf{X} = \begin{bmatrix} 3 \\ 1 \\ 2 \end{bmatrix} \quad \Longrightarrow \quad \widehat{\mathbf{x}} = (1.5,\, 0.5)^{\top}.$$

The Euclidean reprojection error is

$$\begin{aligned} e &= \|\widehat{\mathbf{x}} - \mathbf{x}_m\|_2 = \sqrt{(1.5 - 280)^2 + (0.5 - 200)^2} \\ &= \sqrt{278.5^2 + 199.5^2} = \sqrt{117362.5} \approx \boxed{342.5821 \text{ px}}. \end{aligned}$$

Python code

experiments/assignment1/q9_reprojection_error.py

```python
"""Q9: use exact homogeneous projection and Euclidean pixel error."""
import sympy as sp


def reprojection_error():
    P = sp.Matrix([[1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 1, 0]])
    world = sp.Matrix([3, 1, 2, 1])
    measured = sp.Matrix([280, 200])
    image_h = P * world
    projected = image_h[:2, 0] / image_h[2]
    residual = projected - measured
    squared = residual.dot(residual)
    return projected, residual, squared, sp.sqrt(squared)


if __name__ == "__main__":
    projected, residual, squared, error = reprojection_error()
    print("Projected image point:", tuple(projected))
    print("Residual (predicted - measured):", tuple(residual))
    print("Squared error (px^2):", squared)
    print(f"Euclidean reprojection error (px): {float(error):.10f}")
```

Run output

```
Projected image point: (3/2, 1/2)
Residual (predicted - measured): (-557/2, -399/2)
Squared error (px^2): 234725/2
Euclidean reprojection error (px): 342.5821069466
```

Question 10

Solution

Unit-scale orthographic projection along $+Z$ discards the depth coordinate:

$$\mathbf{x} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 5 \\ 4 \\ 8 \end{bmatrix} = \boxed{\begin{bmatrix} 5 \\ 4 \end{bmatrix}}.$$

Python code

experients/assignment1/q10_orthographic.py

```python
"""Q10: unit-scale orthographic projection along +Z."""
import sympy as sp


def orthographic_projection():
    projection = sp.Matrix([[1, 0, 0], [0, 1, 0]])
    point = sp.Matrix([5, 4, 8])
    return projection * point


if __name__ == "__main__":
    image = orthographic_projection()
    print("Orthographic image point:", tuple(image))
```

Run output

```
Orthographic image point: (5, 4)
```

Reproducibility

To rerun the calculations and build the report from the repository root:

```
python -m pip install -r experients/assignment1/requirements.txt
powershell -NoProfile -File report/build.ps1
```

Each question has a standalone script using NumPy and/or SymPy. The build runs all ten scripts, captures their standard output, and includes the complete source and output directly under the corresponding question.

The independent checks in `experients/assignment1/verify.py` cover all ten scripts. They also confirm the exact DLT rank and null vector, the two-view reprojection, distance preservation under 100 proper rotations, and the epipolar constraint for independently projected points. All checks passed. The complete verification log is saved in `experients/assignment1/results/verify.txt`.

Recorded execution environment

Component	Recorded value
Python	3.10.16
NumPy	2.2.6
SymPy	1.13.3
Manifest SHA-256 (first 16 hex)	`5942206ffeb81401`

The source files and printed outputs are linked by SHA-256 hashes in `experients/assignment1/results/manifest.json`. The input data are recorded in `experients/assignment1/inputs.md`. Numerical checks use the unrounded arrays; displayed negative zeros are rounding artifacts.